Laboratory Assessment III

PEC-2 Laboratory: Full Stack Development - BIT26PE05

Run a basic Java Maven project

Varad Sushant Deshmukh[123B1F022] TY-IT-A, PCCOE College

AIM:

To configure the H2 in-memory database with a Spring Boot application and perform CRUD (Create, Read, Update, Delete) operations using RESTful APIs.

OBJECTIVE:

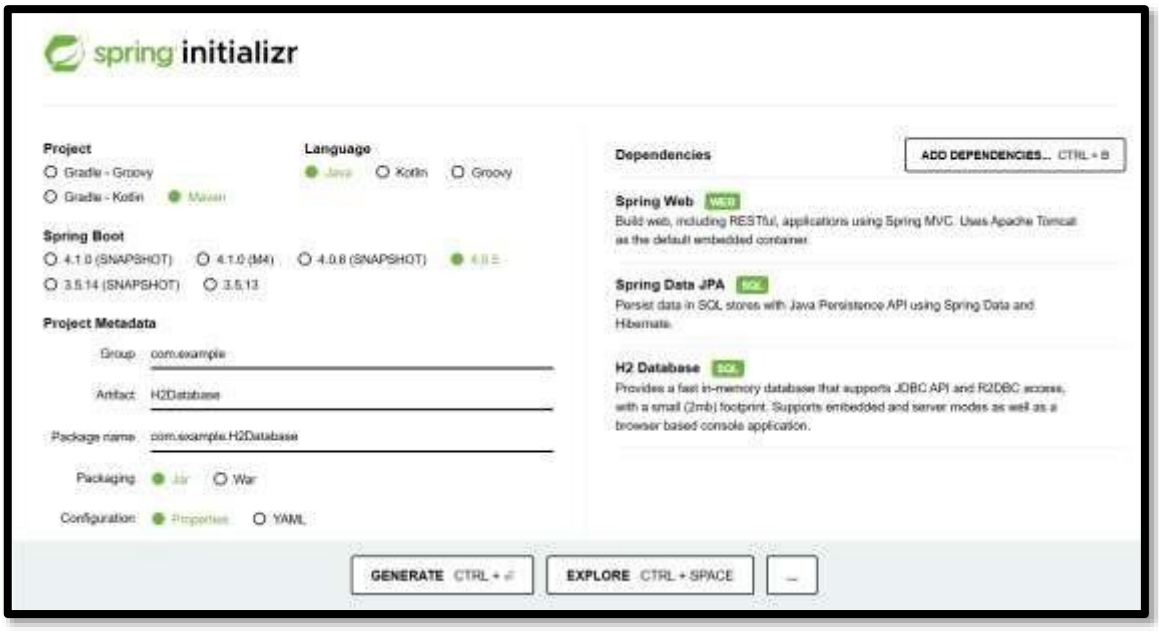
- To integrate H2 database with Spring Boot
- To configure database properties using application.properties
- To create an Entity, Repository, Service, and Controller
- To implement CRUD operations using Spring Data JPA
- To test API endpoints and verify data persistence

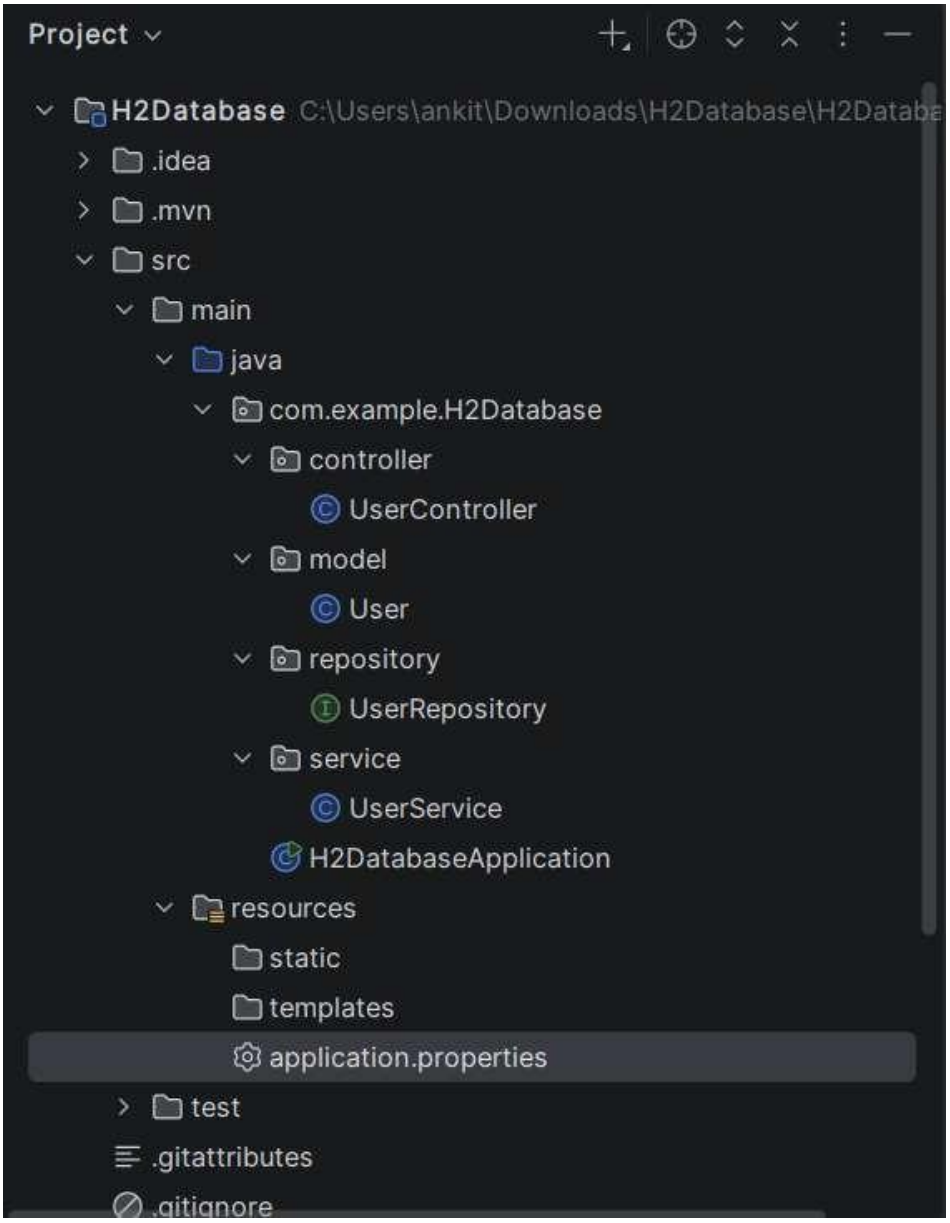
STEP 1: Go to: https://start.spring.io

Select:

- Project: **Maven**
- Language: **Java**
- Dependencies:
    - Spring Web
    - Spring Data JPA
    - H2 Database

Download and open in IntelliJ





STEP 2: Configure H2 Database

Application.properties:

```
spring.application.name=H2Database

server.port=8082

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Controller:

```
package com.example.H2Database.controller;
import com.example.H2Database.model.User;
import com.example.H2Database.service.UserService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/users")
public class UserController {
   private final UserService service;
   public UserController(UserService service) {
      this.service = service;}
   @GetMapping
   public List<User> getUsers() {
      return service.getAllUsers();
   }
   @PostMapping
   public User addUser(@RequestBody User user) {
      return service.saveUser(user);
   }
   @DeleteMapping("/{id}")
   public void deleteUser(@PathVariable int id) {
      service.deleteUser(id); }}
```

Entity:

```
package com.example.H2Database.model;
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;

   private String name;
}
```

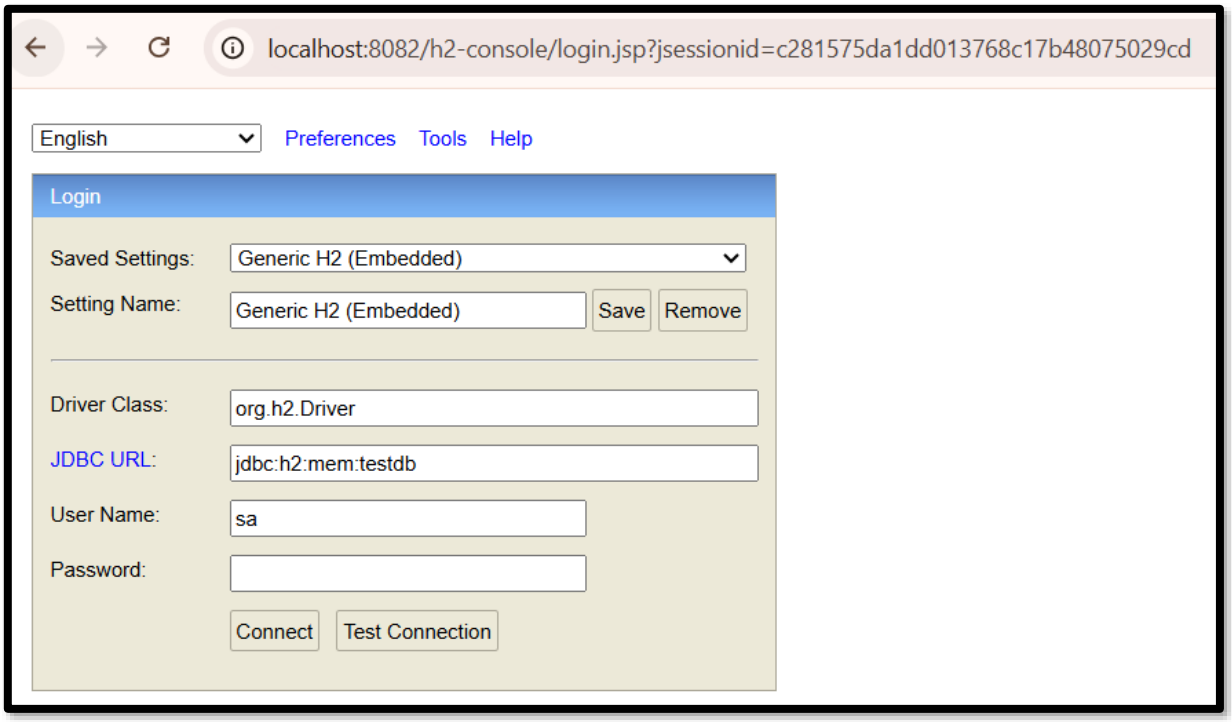
Service:

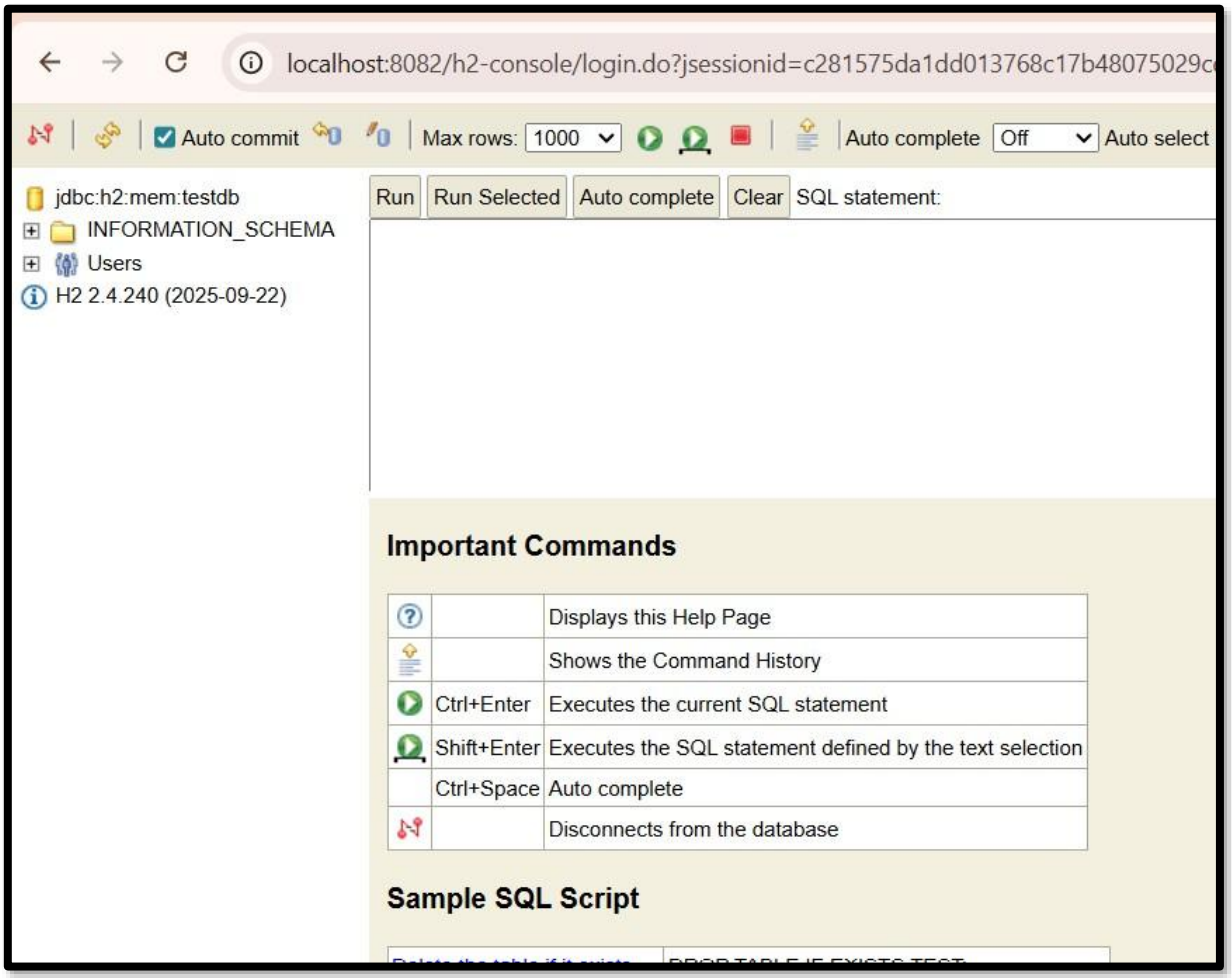
```
package com.example.H2Database.service;
import com.example.H2Database.model.User;
import com.example.H2Database.repository.UserRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {
    private final UserRepository repo;
    public UserService(UserRepository repo) {
        this.repo = repo;
    }
    public List<User> getAllUsers() {
        return repo.findAll();
    }
    public User saveUser(User user) {
        return repo.save(user);
    }
    public void deleteUser(int id) {
        repo.deleteById(id);
    }
}
```

Repository

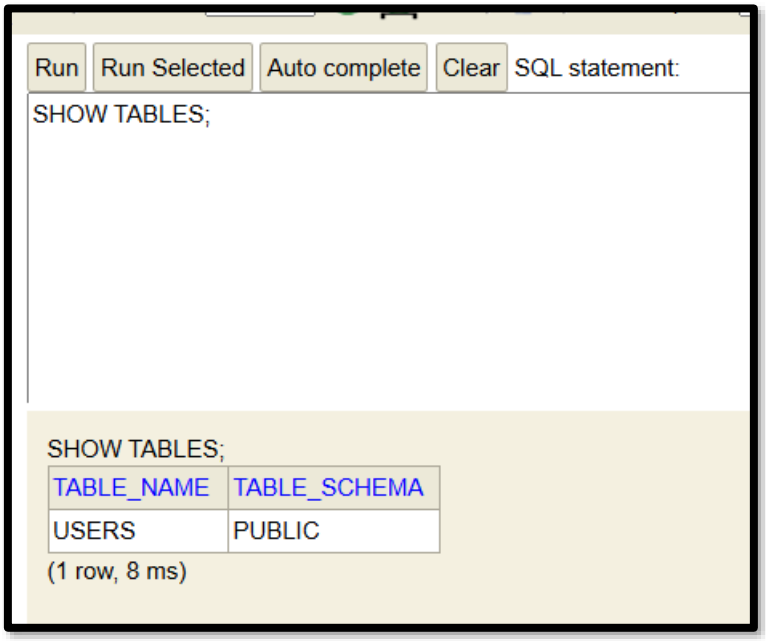
```
package com.example.H2Database.repository;
import com.example.H2Database.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {
}
```

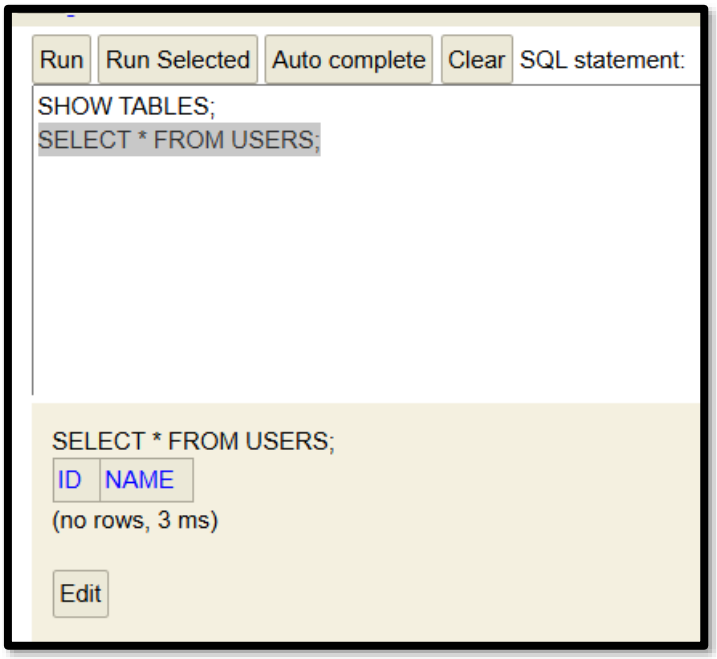
HIT: http://localhost:8082/h2-console


localhost:8082/h2-console/login.jsp?jsessionid=c281575da1dd013768c17b48075029cd
English
Preferences
Tools
Help
Login
Saved Settings:
Generic H2 (Embedded)
Setting Name:
Generic H2 (Embedded)
Save
Remove
Driver Class:
org.h2.Driver
JDBC URL:
jdbc:h2:mem:testdb
User Name:
sa
Password:
Connect
Test Connection


localhost:8082/h2-console/login.do?jsessionid=c281575da1dd013768c17b48075029c
Auto commit
Max rows:
1000
Auto complete
Off
Auto select
jdbc:h2:mem:testdb
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
Run
Run Selected
Auto complete
Clear
SQL statement:
Important Commands
Displays this Help Page
Shows the Command History
Ctrl+Enter
Executes the current SQL statement
Shift+Enter
Executes the SQL statement defined by the text selection
Ctrl+Space
Auto complete
Disconnects from the database
Sample SQL Script

Check table is created





Insert Data (CREATE)

```
INSERT INTO USERS (NAME) VALUES ('John');
INSERT INTO USERS (NAME) VALUES ('Alice');
```

```
INSERT INTO USERS (NAME) VALUES ('John');
Update count: 1
(36 ms)

INSERT INTO USERS (NAME) VALUES ('Alice');
Update count: 1
(0 ms)
```

```
SELECT * FROM USERS;
INSERT INTO USERS (NAME) VALUES ('John');
INSERT INTO USERS (NAME) VALUES ('Alice');
```

```
SHOW TABLES;
```

TABLE_NAME	TABLE_SCHEMA
USERS	PUBLIC

(1 row, 6 ms)

```
SELECT * FROM USERS;
```

ID	NAME
1	John
2	Alice

(2 rows, 1 ms)

```
INSERT INTO USERS (NAME) VALUES ('John');
Update count: 1
(0 ms)

INSERT INTO USERS (NAME) VALUES ('Alice');
Update count: 1
(0 ms)
```

DELETE Record:

```
DELETE FROM USERS WHERE ID=2;
```

```
DELETE FROM USERS WHERE ID=2;
Update count: 1
(9 ms)
```

UPDATE Record:

```
UPDATE USERS SET NAME='Ram' WHERE ID=1;
SELECT * FROM USERS;
```

```
UPDATE USERS SET NAME='Ram' WHERE ID=1;
Update count: 1
(0 ms)

SELECT * FROM USERS;
```

ID	NAME
1	Ram
3	John
4	Alice
5	John
6	Alice

(5 rows, 1 ms)

Conclusion:

The assignment successfully demonstrated the integration of the H2 in-memory database with Spring Boot. CRUD operations were implemented using RESTful APIs and validated through testing. The use of H2 simplified database setup and enabled efficient data handling, providing a clear understanding of backend development and database interaction.